

Assignment #D: Mock Exam

Updated 1955 GMT+8 Dec 5, 2025

2025 fall, Compiled by 窦兆文 元培学院

说明:

1. Dec月考: AC6 (请改为同学的通过数)。考试题目都在“题库 (包括计概、数算题目)”里面, 按照数字题号能找到, 可以重新提交。作业中提交自己最满意版本的代码和截图。
2. 解题与记录: 对于每一个题目, 请提供其解题思路 (可选), 并附上使用Python或C++编写的源代码 (确保已在OpenJudge, Codeforces, LeetCode等平台上获得Accepted)。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。(推荐使用Typora <https://typoraio.cn> 进行编辑, 当然你也可以选择Word。)无论题目是否已通过, 请标明每个题目大致花费的时间。
3. 提交安排: 提交时, 请首先上传PDF格式的文件, 并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像, 提交的文件为PDF格式, 并且“作业评论”区包含上传的.md或.doc附件。
4. 延迟提交: 如果你预计无法在截止日期前提交作业, 请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业, 以保证顺利完成课程要求。

1. 题目

E02734:十进制到八进制

<http://cs101.openjudge.cn/practice/02734>

思路:

代码

```
a = int(input())
result = ""
temp = a
while temp > 0:
    result = str(temp % 8) + result
    temp //= 8
print(result)
```

代码运行截图（至少包含有"Accepted"）

#51211372提交状态

状态: Accepted

源代码

M21509:序列的中位数

heap, <http://cs101.openjudge.cn/practice/21509>

思路:

用大顶堆维护较小的一半数字，堆顶就是中位数

用小顶堆维护较大的一半数字

每次添加元素后调整两个堆的平衡

在索引为偶数（即第1, 3, 5...个数）时输出中位数

代码

```
import heapq

n = int(input())
a = list(map(int, input().split()))
max_heap = []
min_heap = []

for i in range(n):
    num = a[i]
    if not max_heap or num <= -max_heap[0]:
        heapq.heappush(max_heap, -num)
    else:
        heapq.heappush(min_heap, num)
    if len(max_heap) > len(min_heap) + 1:
        heapq.heappush(min_heap, -heapq.heappop(max_heap))
    elif len(min_heap) > len(max_heap):
        heapq.heappush(max_heap, -heapq.heappop(min_heap))
    if i % 2 == 0:
        print(-max_heap[0])
```

代码运行截图（至少包含有"Accepted"）

#51211407提交状态

状态: **Accepted**

源代码

M27306: 植物观察

disjoint set, bfs, <http://cs101.openjudge.cn/practice/27306/>

思路:

如果两个植物属于同一类，它们之间的关系是"相同"(0)

如果两个植物属于不同类，它们之间的关系是"不同"(1)

使用带权并查集来维护每个节点到根节点的关系:

relation[i] = 0 表示节点 i 与其父节点同类

relation[i] = 1 表示节点 i 与其父节点不同类

代码

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.relation = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            root = self.find(self.parent[x])
            self.relation[x] = (self.relation[x] + self.relation[self.parent[x]]) % 2
            self.parent[x] = root
        return self.parent[x]

    def union(self, x, y, rel):
        root_x = self.find(x)
        root_y = self.find(y)

        if root_x == root_y:
            return (self.relation[x] ^ self.relation[y]) == rel
        else:
            self.parent[root_y] = root_x
            self.relation[root_y] = (self.relation[x] ^ rel ^ self.relation[y])
            return True

n, m = map(int, input().split())
uf = UnionFind(n)

valid = True
for _ in range(m):
    x, y, rel = map(int, input().split())
    if not uf.union(x, y, rel):
        valid = False
        break

print("YES" if valid else "NO")
```

代码运行截图（至少包含有"Accepted"）

	题目	排名	状态	提问
--	----	----	----	----

#51211433提交状态

状态: Accepted

M29740:神经网络

Topological order, <http://cs101.openjudge.cn/practice/29740/>

思路:

代码

代码运行截图（至少包含有"Accepted"）

T27351:01最小生成树

mst, <http://cs101.openjudge.cn/practice/27351/>

思路:

由于是完全图，任意两点间都有边：

明确给出的 m 条边权重为 1

其余所有边权重为 0

在最小生成树中，我们优先选择权重为 0 的边

只有当必须使用权重为 1 的边时才使用

问题转化为：用最少的权重为1的边，使得所有点连通

将权重为1的边看作"禁止边"或"昂贵边"

先尽可能使用权重为0的边连接节点

统计需要多少条权重为1的边才能让图连通

对于每个连通分量，找出它能通过权重为0的边连接到哪些其他连通分量

使用并查集维护连通性

BFS/DFS 找出所有通过"非禁止边"能到达的点

代码

```

from collections import deque

class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py:
            return False
        if self.rank[px] < self.rank[py]:
            px, py = py, px
        self.parent[py] = px
        if self.rank[px] == self.rank[py]:
            self.rank[px] += 1
        return True

    def count_components(self, n):
        return len(set(self.find(i) for i in range(n)))

n, m = map(int, input().split())
forbidden = set()
for _ in range(m):
    a, b = map(int, input().split())
    a -= 1
    b -= 1
    forbidden.add((min(a, b), max(a, b)))

uf = UnionFind(n)
unvisited = set(range(n))

while unvisited:
    start = next(iter(unvisited))
    queue = deque([start])
    unvisited.remove(start)
    component = [start]

    while queue:

```

```

u = queue.popleft()
to_remove = []
for v in unvisited:
    edge = (min(u, v), max(u, v))
    if edge not in forbidden:
        to_remove.append(v)
        queue.append(v)
        component.append(v)

for v in to_remove:
    unvisited.remove(v)
for i in range(1, len(component)):
    uf.union(component[0], component[i])
num_components = uf.count_components(n)
result = num_components - 1

print(result)

```

代码运行截图（至少包含有"Accepted"）

#51211488提交状态

状态: **Accepted**

源代码

T30193:哈密顿激活层

DFS+剪枝, <http://cs101.openjudge.cn/practice/30193/>

思路:

连通性剪枝:

在每次移动前, 使用BFS检查从下一个位置是否能访问到所有剩余格子
如果可达格子数 < 需要访问的格子数, 直接剪枝

启发式搜索顺序:

优先尝试靠近下一个约束点的方向

使用曼哈顿距离作为启发函数

约束索引优化：

预先将约束按时间排序

动态追踪当前应该满足哪个约束

更早的剪枝检查：

在尝试每个方向前就进行连通性检查

避免无效的递归

代码

```

def solve():
    n, m, k, b = map(int, input().split())
    constraints = {}
    start_pos = None
    for _ in range(k):
        r, c, t = map(int, input().split())
        constraints[(r, c)] = t
        if t == 1:
            start_pos = (r, c)
    obstacles = set()
    for _ in range(b):
        r, c = map(int, input().split())
        obstacles.add((r, c))
    total_cells = n * m - b
    sorted_constraints = sorted([(t, r, c) for (r, c), t in constraints.items()])
    visited = [[False] * (m + 1) for _ in range(n + 1)]
    for r, c in obstacles:
        visited[r][c] = True

    path = []
    directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]

    def is_valid(r, c):
        return 1 <= r <= n and 1 <= c <= m and not visited[r][c]

    def count_reachable(r, c):
        count = 0
        queue = [(r, c)]
        seen = {(r, c)}
        idx = 0
        while idx < len(queue):
            cr, cc = queue[idx]
            idx += 1
            if not visited[cr][cc]:
                count += 1
            for dr, dc in directions:
                nr, nc = cr + dr, cc + dc
                if (nr, nc) not in seen and 1 <= nr <= n and 1 <= nc <= m and not visited[nr][nc]:
                    seen.add((nr, nc))
                    queue.append((nr, nc))
        return count

    def dfs(r, c, step, constraint_idx):

```

```

if (r, c) in constraints and constraints[(r, c)] != step:
    return False

visited[r][c] = True
path.append((r, c))
if len(path) == total_cells:
    return True

if constraint_idx < len(sorted_constraints) and sorted_constraints[constraint_idx][0] ==:
    constraint_idx += 1
if constraint_idx < len(sorted_constraints):
    next_time, next_r, next_c = sorted_constraints[constraint_idx]
    dist = abs(next_r - r) + abs(next_c - c)
    time_left = next_time - step
    if dist > time_left or (time_left - dist) % 2 != 0:
        visited[r][c] = False
        path.pop()
        return False

remaining = total_cells - len(path)
next_moves = []
for dr, dc in directions:
    nr, nc = r + dr, c + dc
    if is_valid(nr, nc):
        priority = 0
        if constraint_idx < len(sorted_constraints):
            _, next_r, next_c = sorted_constraints[constraint_idx]
            priority = abs(next_r - nr) + abs(next_c - nc)
        next_moves.append((priority, nr, nc))

next_moves.sort()

for _, nr, nc in next_moves:
    if remaining > 1:
        reachable = count_reachable(nr, nc)
        if reachable < remaining:
            continue

    if dfs(nr, nc, step + 1, constraint_idx):
        return True
visited[r][c] = False
path.pop()
return False
if start_pos and dfs(start_pos[0], start_pos[1], 1, 0):

```

```
        for r, c in path:
            print(r, c)
    else:
        print(-1)

solve()
```

代码运行截图（至少包含有"Accepted"）

#51211556提交状态

状态: **Accepted**

2. 学习总结和收获

如果作业题目简单，有否额外练习题目，比如：OJ“计概2025fall每日选做”、CF、LeetCode、洛谷等网站题目。

好难，对期末机考还得练练